

Reviewer Pack — Tropical Derivation Sparsity Limits Phase Cell Count

Entry f7d5775dddc6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 07:04:32 UTC

Tropical Derivation Sparsity Limits Phase Cell Count

Entry ID: f7d5775dddc6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 07:04:32 UTC

1. Conjecture

Field A (mathematical branch): Tropical Circuit Weight Analysis (TCWA) — Bounded Arithmetic

Field B (complexity object): Polynomial Hierarchy (PH)

Statement:

For any tropical circuit C computing a function provably total in $\text{I}\Delta_0$, the number of phase cells in its circuit phase space is bounded above by a polynomial in the sparsity of its tropical derivation $\delta(C)$, where sparsity is the maximum number of non-zero entries in the active edge structure across all inputs.

Rationale (proposer's reasoning):

This conjecture tests Axiom A1 (Phase Stability Axiom) by linking phase cell complexity — a geometric proxy for proof complexity — to the combinatorial sparsity of the tropical derivation. Since tropical derivations capture the sensitivity of computation paths to input perturbations, their sparsity reflects how 'stable' a phase is under small changes. In $\text{I}\Delta_0$, functions have limited induction strength, so their tropicalizations should not induce exponentially many phase transitions. If true, this would imply that functions with sparse derivation maps cannot generate complex phase geometries, reinforcing the idea that $\text{I}\Delta_0$ corresponds to low phase complexity in TCWA.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time:)

Framework membership: framework fw_a1a152ae17, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cfc8b8a5ce4b5cbd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Polynomial bound on phase cell count by tropical derivation sparsity

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	UNCERTAIN	0.90	SAFE	HITS
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical circuit weight analysis AND polynomial hierarchy AND bounded arithmetic - sparsity of tropical derivation AND phase cell count AND ID_0 - tropical circuit phase space AND polynomial bounds AND Δ_0 provable total functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Define the tropical circuit and derivation sparsity calculation
    def tropical_circuit(n):
        return sum([random.choice([-1, 0, 1]) * x for x in range(1, n + 1)])

    def derivation_sparsity(circuit):
        active_edges = [x != 0 for x in circuit]
        return max(active_edges.count(True) for _ in range(10))

    # Define the phase cell count calculation
    def phase_cell_count(circuit):
        inputs = [[i] * n for i in range(-5, 6)]
        cells = set()
        for input_val in itertools.product(*inputs):
            output = tropical_circuit(input_val)
            cells.add(output)
        return len(cells)

    # Generate a random tropical circuit
    n = random.randint(5, 10)
    circuit = [random.choice([-1, 0, 1]) * x for x in range(1, n + 1)]

    # Calculate the derivation sparsity and phase cell count
    sparsity = derivation_sparsity(circuit)
    cells = phase_cell_count(circuit)
```

```

# Check if the conjecture holds
conjecture_holds = cells <= sparsity ** 2 # Polynomial bound of degree 2

return {
    "metric_name": "Phase Cell Count",
    "metric_value": cells,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Counterexample: n={n}, sparsity={sparsity}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_cells = sum(r["metric_value"] for r in results) / len(results)
    std_cells = math.sqrt(sum((r["metric_value"] - mean_cells) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_cells} std={std_cells} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_cells} std={std_cells} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5f9ada46.py", line 63, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5f9ada46.py", line 44, in run_trial
    cells = phase_cell_count(circuit)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5f9ada46.py", line 34, in phase_cell_count
    output = tropical_circuit(input_val)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5f9ada46.py", line 23, in tropical_circuit
    return sum([random.choice([-1, 0, 1]) * x for x in range(1, n + 1)])
               ~~~~~

```

TypeError: can only concatenate tuple (not "int") to tuple

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, preventing a conclusive evaluation of the conjecture. | next: Fix the test code to resolve the TypeError and re-run the test to gather data for a conclusive evaluation.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	groq	llama-3.3-70b-versat	0	0	4609
2	novelty	groq	llama-3.3-70b-versat	0	0	4403
3	novelty	groq	llama-3.3-70b-versat	0	0	4522
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10982
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8010
6	judge	groq	llama-3.3-70b-versat	0	0	4560

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 37084 ms total latency. Provider mix: {'groq': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/f7d5775dddc6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f7d5775dddc6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f7d5775dddc6.tar.gz` (if generated)